作业 1：TuGraph 图数据库实验报告

1 实验目标与整体设计

本实验完成 TuGraph 平台部署、图模型建立、CSV 数据导入、增删改查及自定义聚合查询，以疾病、症状、科室和药品为实体：节点保存实体属性，边表示实体关联，Cypher 用于模式匹配与统计。

整体流程为：UTF-8 CSV → JSON 图模型和字段映射 → lgraph_import 原生导入 → TuGraph 存储引擎 → Browser 查询交互。节点用 name 作为主键，关系文件按主键连接节点。原始 CSV 与数据库目录分别保存，便于复现和备份。

运行环境为 Windows Docker Desktop 的 Linux 容器，数据库来自官方发布的 4.5.1 RPM 包。最终容器使用 Debian 13 基础环境，宿主机 HTTP 17070、Bolt 17687、RPC 19090 映射至容器对应端口，映射地址限定为 127.0.0.1。



图 1 TuGraph Browser 登录结果（medical_demo：17 个节点、20 条关系）

2 安装、部署与启动

从 TuGraph 官方 GitHub Release 下载 4.5.1 RPM 安装包。在容器中安装 rpm2cpio、cpio、libgomp1、libgfortran5 后解包二进制和 Browser 资源，服务成功启动。Dockerfile 固定基础镜像 digest，入口替换为 lgraph_server。

在项目目录启动 Docker Desktop 后，执行 setup.ps1，构建镜像、导入 CSV 数据并启动服务。启动的核心命令为：

```
docker run -d --name tugraph-assignment --restart unless-stopped `
  -p 127.0.0.1:17070:7070 -p 127.0.0.1:17687:7687 `
  -p 127.0.0.1:19090:9090 `
  --mount "type=bind,source=${PWD},target=/workspace" `
  tugraph-assignment-clean:4.5.1

docker ps --filter name=tugraph-assignment
docker logs --tail 30 tugraph-assignment
```

服务读取 `config/lgraph.json`，数据库位于 `/workspace/runtime/db`。浏览器访问 `http://127.0.0.1:17070`，填写 Bolt 地址 127.0.0.1:17687，使用账号 admin、默认密码 73@TuGraph 登录，进入 medical_demo 图项目。

启动日志显示 TuGraph Graph Database v4.5.1，包含以下成功信息；完整原始输出保存在 `evidence/startup.log`：

```
Listening for RPC on port 9090
Listening for REST on port 7070
bolt server run
Server started.
```

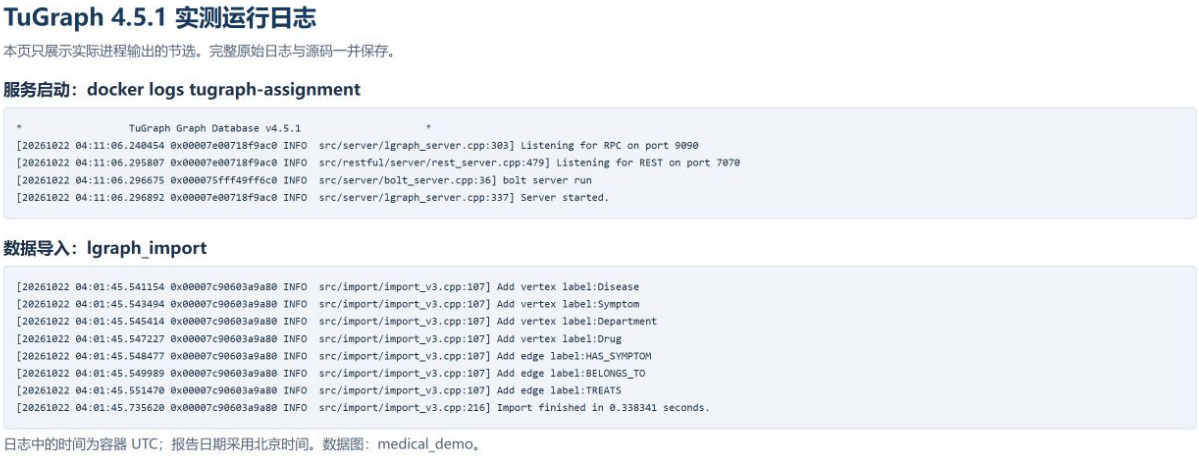


图 2 TuGraph 服务启动与 CSV 数据导入日志

停止和再次启动分别执行 docker stop tugraph-assignment、docker start tugraph-assignment。数据通过挂载目录保存，重启容器后再次核对的数量与导入结果一致。

3 图模型与数据导入

3.1 属性图模型

节点类型	主键	其他属性	数量
Disease	name: STRING	category: STRING	4
Symptom	name: STRING	severity: STRING	7
Department	name: STRING	无	3
Drug	name: STRING	kind: STRING	3

边类型	起点	终点	数量
HAS_SYMPTOM	Disease	Symptom	12
BELONGS_TO	Disease	Department	4
TREATS	Disease	Drug	4

TREATS 连接疾病与对应药品。



图 3 医疗知识图谱模型

3.2 CSV 导入与数量核对

准备四份节点文件 diseases.csv、symptoms.csv、departments.csv、drugs.csv，以及三份关系文件 disease_symptom.csv、disease_department.csv、disease_drug.csv。文件使用 UTF-8 编码，首行为表头；import.json 的 header 设为 1。

import.json 中 schema 定义标签、属性、主键和边约束，files 定义文件路径与列映射。例如疾病症状边的映射如下：

```
{
  "path": "/workspace/data/disease_symptom.csv",
  "header": 1, "format": "CSV", "label": "HAS_SYMPTOM",
  "SRC_ID": "Disease", "DST_ID": "Symptom",
  "columns": ["SRC_ID", "DST_ID"]
}
```

在空数据库目录执行原生离线导入：

```
lgraph_import --config_file /workspace/data/import.json \
  --dir /workspace/runtime/db --graph medical_demo
```

导入日志显示四个 vertex label 和三个 edge label 均创建成功，结束信息为 Import finished in 0.338341 seconds。启动服务后，再通过 Cypher 分类型核对导入数量。



图 4 数据导入后各类型节点数量

4 增删改查

在 medical_demo 图的“语句查询”页面逐条执行 Cypher，使用主键为“临时症状-CRUD”的节点演示，再将其删除。

4.1 新增与查询

```
CREATE (x:Symptom {name:'临时症状-CRUD', severity:'测试'})
RETURN x.name AS name, x.severity AS severity;

MATCH (x:Symptom {name:'临时症状-CRUD'})
RETURN x.name AS name, x.severity AS severity;
```



图 5 创建症状节点



图 6 查询新增节点

4.2 修改与删除

```
MATCH (x:Symptom {name:'临时症状-CRUD'})
SET x.severity = '已修改'
RETURN x.name AS name, x.severity AS severity;

MATCH (x:Symptom {name:'临时症状-CRUD'}) DELETE x;
```



图 7 修改节点的 severity 属性



图 8 删除节点的执行结果

4.3 删除后复查

```
MATCH (x:Symptom {name:'临时症状-CRUD'})
RETURN count(x) AS remaining;
```



图 9 删除后复查结果：remaining 为 0

5 自定义聚合查询

本实验统计每种疾病关联的不同症状数量。MATCH 匹配疾病到症状的路径，按疾病分组，用 count(DISTINCT s) 去重计数，再按数量降序排序。

```
MATCH (d:Disease)-[:HAS_SYMPTOM]->(s:Symptom)
RETURN d.name AS disease, count(DISTINCT s) AS symptom_count
ORDER BY symptom_count DESC;
```



图 10 疾病症状数量聚合查询结果

5.1 用户交互与关系图

在 Browser 中选中 medical_demo，输入 Cypher 后点击“执行”，可切换“表格文本”和“点边视图”。以下查询显示疾病与症状的关系网络：

```
MATCH (d:Disease)-[r:HAS_SYMPTOM]->(s:Symptom)
RETURN d,r,s;
```

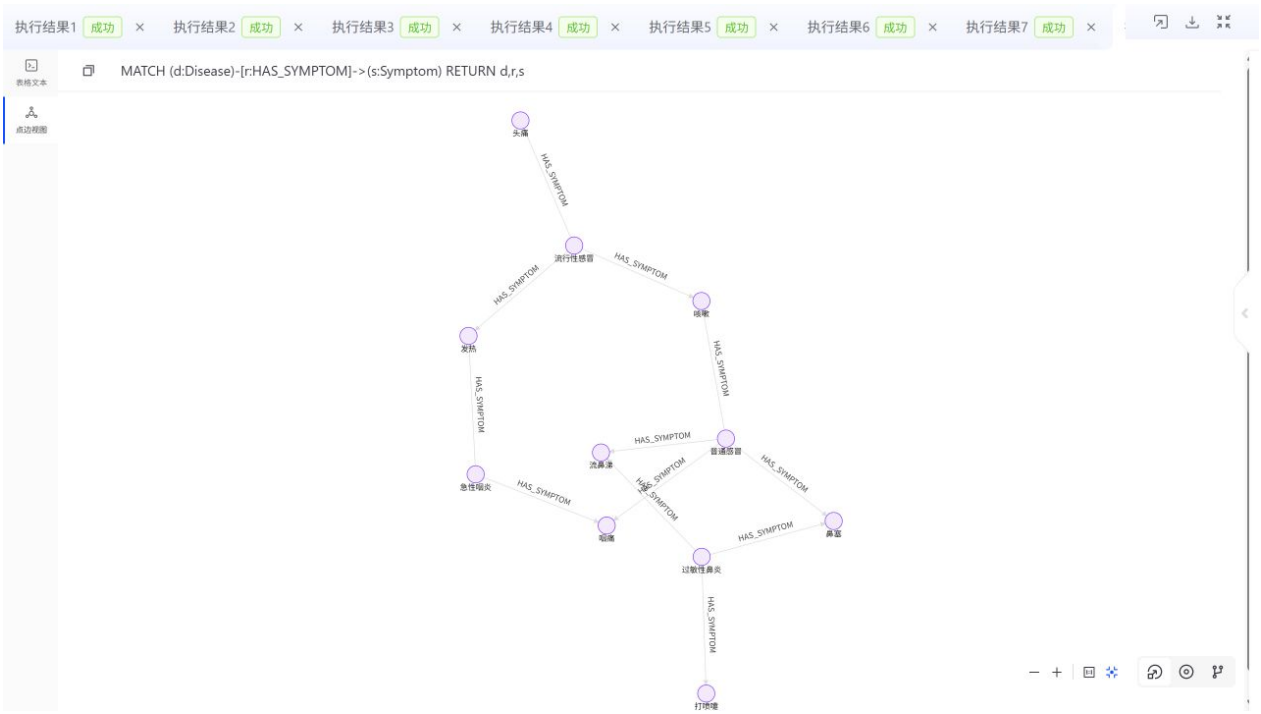


图 11 疾病与症状的关系图

6 实验结果与总结

本实验完成了 TuGraph 4.5.1 的安装与启动，建立医疗知识图谱模型，导入 7 个 CSV 文件，并执行了节点增删改查、关系查询和聚合统计。实验结果如下。

核对项	实测结果
TuGraph 版本 / 图项目	4.5.1 / medical_demo
节点数 / 节点类型	17 / 4
关系数 / 关系类型	20 / 3
CRUD 临时节点剩余数	0
聚合查询	普通感冒 4、过敏性鼻炎 3、流行性感冒 3、急性咽炎 2
服务重启后核对	与上述数量一致

实验结果表明，TuGraph 能够保存疾病、症状、科室和药品之间的关联，并通过 Cypher 完成路径匹配、数据维护及分组统计。新增节点经修改和删除后，图中节点数恢复为 17，关系数保持为 20。重启服务后，查询结果与导入数据一致。

项目代码：`https://github.com/MuonChaser/tugraph-homework-1`。